

TRSI – Homework 3

Experimental Results

Dataset

The experiments were done on RailSem19 dataset, which contains 8500 pairs of pictures and segmentation masks taken from a train / tram in some scenarios, “available thanks to a large online community of rail enthusiasts who upload driver’s cabin”.

The segmentation masks contains gray pixels ranging from 0 to 18 + 255, the significations of each pixels is the following: the first 19 pixels represents the following classes: [*road, sidewalk, construction, tram-track, fence, pole, traffic-light, traffic-sign, vegetation, terrain, sky, human, rail-track, car, truck, trackbed, on-rails, rail-raised, rail-embedded*], and the pixels with the value 255 represents the class *void*.

The dataset can be downloaded from here: <https://www.wilddash.cc/railsem19>. To obtain it, an approve from the authors is required.

Experiments setup

→ As a preprocessing step, the RailSem19 segmentation pairs were taken (image – segmentation mask, 1920 x 1080), a central crop was taken (so the images were 1080 x 1080), and then they were resized to 512 x 512 (using cv2.INTER_AREA for the images and cv2.INTER_NEAREST for the masks), and this gave the sets: images and folders, the images names were ‘rs{idx:05f}.jpg’, and the corresponding masks were ‘rs{idx:05f}.png’.

→ The original color scheme of the segmentation masks were the grays one, I tried to insert in some kind of way the signification of the labels into the masks to try to improve the performance, my color scheme was obtained by passing the class labels through word2vec trained on Google News, then passing the vectors through a PCA to obtained 3 values, then mapped those values to [0, 1], then associated a pixel which each ones, then applied these color mappings into the dataset’s masks. The experiments proved that these color schemes didn’t have that much of an impact on the results.

→ Other kind of controls were also tried, such as Canny Edge (the one from opencv, with the thresholds 100 and 200) applied on each image, and the combination of the two controls (Canny edge + Segmentation mask, on different channels, different orderings of the channels were tried, but the ordering didn’t have some huge impact on the results).

→ In the ControlNet paper, they don’t mention about training with or without text prompts, only on the inference step they experiment with / without text prompts. We experimented with more types of prompts (no prompt, fixed prompt), but the RailSem19 dataset didn’t have any text descriptions of the photos, so the BLIP2 model was used to generate some as a preprocessing step, the inspiration of the usage of this image-to-description model was this notebook: https://colab.research.google.com/github/huggingface/community-events/blob/main/jax-controlnet-sprint/dataset_tools/create_pose_dataset.ipynb released by MediaPipe team from Google and the Diffusers team from HuggingFace. In addition to that, a “default

part” was appended to each prompt (“, high quality, extremely detailed, 4K, HQ”), they used a similar string in the ControlNet paper.

→ There was an option to also give negative prompts as a parameter, but only on the inference step (in the training pipeline there is no explicit parameter `negative_prompts`). Some negative prompts were gathered from this web article: <https://openaijourney.com/stable-diffusion-negative-prompts/>, they seemed to work fine (and to improve a bit the performance of the model).

```
negative_prompt = (
    "\'accidents, derailments, overgrown tracks, overgrown stations, impossible curves, missing switches, "
    "cartoon, cgi, render, 3d, artwork, illustration, 3d render, cinema 4d, artstation, octane render, painting, "
    "oil painting, anime, 2d, sketch, drawing, bad photography, bad photo, deviant art, overexposed, simple background, "
    "plain background, grainy, portrait, grayscale, monochrome, underexposed, low contrast, low quality, dark, distorted, "
    "white spots, deformed structures, macro, multiple angles, nsfw, uncensored, nude, ugly, bad anatomy, bad proportions, "
    "blurry, cropped, deformed, disconnected limbs, out of frame, out of focus, dehydrated, error, disfigured, "
    "disgusting, extra arms, extra limbs, extra hands, fused fingers, gross proportions, long neck, low res, jpeg, "
    "jpeg artifacts, malformed limbs, mutated, mutated hands, mutated limbs, missing arms, missing fingers, picture frame, "
    "poorly drawn hands, poorly drawn face, text, signature, username, watermark, worst quality, collage, pixel, pixelated\""
)
```

→ The dataset was split into training (the first 6800 images) and validation (the last 1700 images), so 80% training, 20% validation.

→ For the environment where the experiments took place, my experiments were done on a GTX 1080Ti graphics card, I needed to adjust the hyperparams in such a way to not crash this graphics card, each experiment took ~20 hours, an experiment consisting in 22100 steps (13 epochs on the training set of RailSem19 – 6800 images, with batch size 4).

Metrics

The main performance metric used so far is FID (Fréchet Inception Distance), which is one of the main metrics used to assess the quality of images created by a generative model. Unlike inception score (IS), which measures mainly the distribution of the generated images, FID compares the distribution of the generated images with the ones from the real images (“ground truth”).

For two probability distributions over $\mathbb{R}^n$ having finite mean and variances, their Fréchet distance is:

$$d_F(\mu, \nu) := \left(\inf_{\gamma \in \Gamma(\mu, \nu)} \int_{\mathbb{R}^n \times \mathbb{R}^n} \|x - y\|^2 d\gamma(x, y) \right)^{1/2},$$

For two multidimensional Gaussian distributions $\mathcal{N}(\mu, \Sigma)$ and $\mathcal{N}(\mu', \Sigma')$, FID is explicitly solvable as:

$$d_F(\mathcal{N}(\mu, \Sigma), \mathcal{N}(\mu', \Sigma'))^2 = \|\mu - \mu'\|_2^2 + \text{tr}\left(\Sigma + \Sigma' - 2(\Sigma\Sigma')^{\frac{1}{2}}\right)$$

This allows to define the FID in pseudocode form:

INPUT a function $f : \Omega_X \rightarrow \mathbb{R}^n$.

INPUT two datasets $S, S' \subset \Omega_X$.

Compute $f(S), f(S') \subset \mathbb{R}^n$.

Fit two gaussian distributions $\mathcal{N}(\mu, \Sigma), \mathcal{N}(\mu', \Sigma')$, respectively for $f(S), f(S')$.

RETURN $d_F(\mathcal{N}(\mu, \Sigma), \mathcal{N}(\mu', \Sigma'))^2$.

In most practical cases (also in my work), f is chosen as an Inception v3 model trained on ImageNet, but without its final classification layer, which outputs a 2048-dimensional activation vector of its last pooling layer. S is the original dataset (ground truth), and S' is the dataset formed by the generated images.

Results

First set of experiments

→ In the first set of experiments, I experimented with different kind of parameters, to see what could be obtain. I experimented with the learning_rate a bit and obtained very different results. You can see the results here: <https://wandb.ai/rasvan/rail-scenes-generation-controlnet/reports/Experiments-0--Vmldzo3NzkxNDIx?accessToken=3k7eutuk0pi22umvgc5gc9uaips1eme4301leofmtw1mpiqu6zi5902m9jz537ca>.

→ The experiment with the biggest learning rate crashed (from nans), so there is a limit on that side. I found that the experiments with the learning rate $1e-5$ looked the best in the proposed set up.

→ I may also add that these experiments were done with the first set of BLIP2 prompts (with the Salesforce/blip2-flan-t5-xl model, which is the smaller one – “*meta.jsonl*”). In the next set of experiments, we changed these prompts to some news ones, from the Salesforce/blip2-opt-2.7b-coco model (“*meta_new.jsonl*” for those ones, “*meta_new2.jsonl*” for those with the default prompts appended to them), which was bigger and gave better results. The version trained on the COCO dataset was used, because that dataset also contains a lot of images in some driving scenarios.

Second set of experiments

→ In the second set of experiments, I tried to compare different controls and different types of prompts. There are some results: <https://api.wandb.ai/links/rasvan/k3qqpiy5>.

→ In these experiments, the results were more consistent also because in these ones the seed was fixed, and there were small changes.

→ The results seemed better, but this was also because the masks were a bit easier, not containing any cars or some other complicated obstacles.

→ The best-looking results were the ones from the simple control (only masks) and the new BLIP2 prompts.

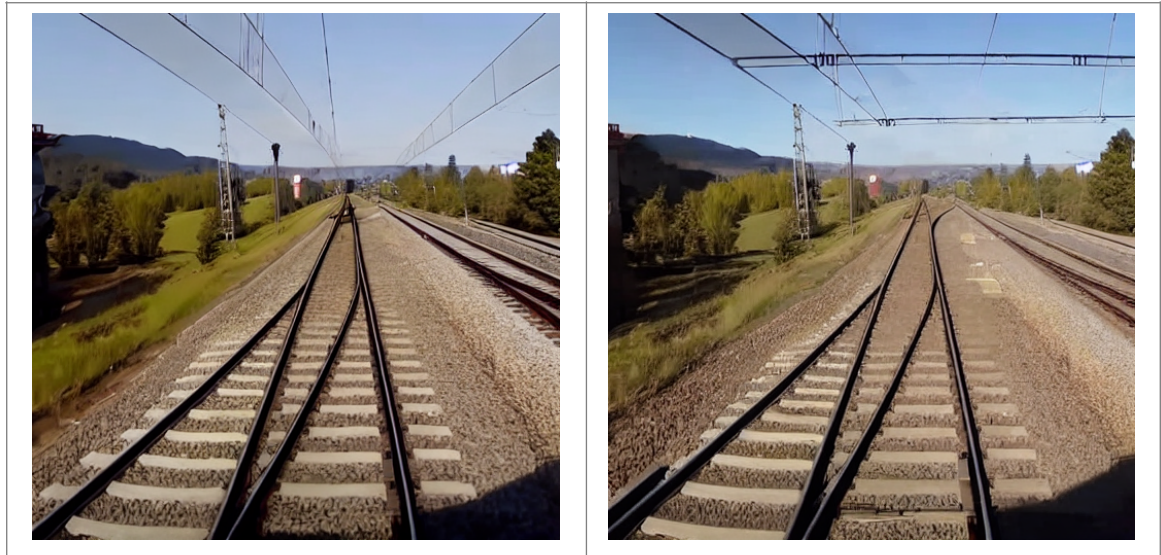
→ In all these experiments, negative prompts were also used in the validation images.

Other experiments

→ Some other things we experimented with were the effect of the noise in the outputs, and how different latents affect the final output during the inference step. Two experiments were done to explore this idea:

- How the same model handles different controls with the same latents – we can see how the image is more refined as the “level of control” increases, so more control leads to better images (which we could see also during the training process).

no prompt	<i>“train tracks high-quality image”</i>
	
<i>“a view of a train track with a train on it, high quality, extremely detailed, 4K, HQ”</i>	<i>“a view of a train track with a train on it, high quality, extremely detailed, 4K, HQ” + negative prompts</i>



- How the same model handles same control with different latents (an initial latent was chosen + 256 random values in the latent space which took values from -1 to 1 with a step of 0.2) – we could see how the chosen values changes the luminosity of the background and some things at the wires.



- The model for the experiments above was v1_0 from there: <https://api.wandb.ai/links/rasvan/k3qqpiy5>.

Metrics Measurements

FID Score	Original Mask	Canny Edges
No prompt	76.70	80.07
Fixed prompt	77.77	79.19
BLIP2	76.72	78.93
Negative prompts	77.12	81.37
Fixed + Negative	77.28	80.95
BLIP2 + Negative	76.57	81.35

So, the metrics proved what we expected from the visual results, so the fact that the BLIP2 + Negative prompts results looked the best. We could also see that in some cases the negative prompts negatively impacted the results, and in others they positively impacted them. We could also see that by adding the extra control (the canny edges), the model performed worse.

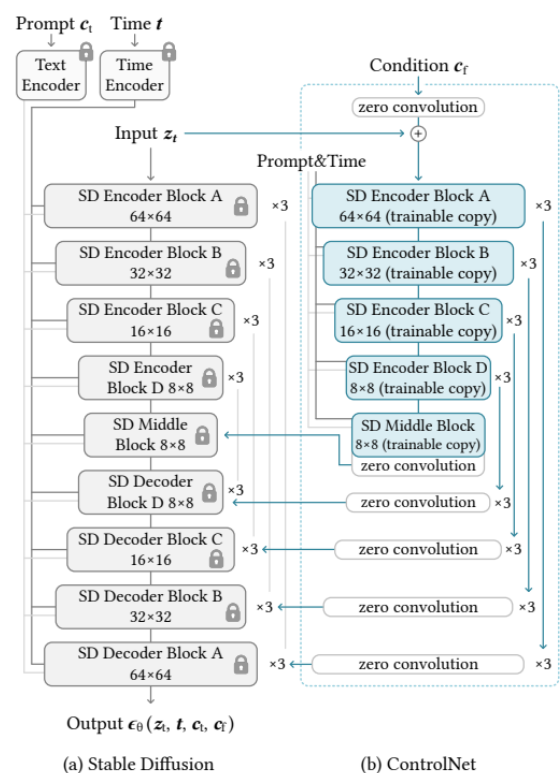
Methodology

Understanding ControlNet – the architecture

ControlNet is a **HyperNetwork** (a smaller network trained to influence the weights of a larger network) applied on some Latent Diffusion Model (Stable Diffusion in this case).

Stable Diffusion's architecture is like a U-Net, having an encoder, a middle block and a skip-connected decoder. Because it's a text-to-image model, they use CLIP as their text encoder. In addition, the diffusion timesteps are encoded with a time encoder using positional encoding (to tell the network how "far" the current input is from the real output).

To build the ControlNet model, the authors locked the "production-ready model" and took half of the encoding blocks and 1 middle block from the Stable Diffusion as a trainable copy (so their weights were initialized with the original model's weights). They also pass the condition images through some network to convert the pixel-space



images into smaller latent images (this network is called $E(\cdot)$ in the original paper and is composed by four convolution layers with 4×4 kernels and 2×2 strides, activated by ReLU). The ControlNet is connected to the locked model through some **zero convolution layers** ($Z(\cdot; \cdot)$, which is an 1×1 convolution layer with both weight and bias initialized to zeros). The reason the layers are initialized with zeros is **to avoid harmful noise influencing the hidden states** of the neural network layers in the trainable copy when the training starts.

Loss function

The objective of the machine learning model ϵ_θ that computes the reverse diffusion process is to predict correctly the noise added from timestep $t-1$ to timestep t given the timestep t and the noisy image x_t . The loss function is:

$$L_{\text{DM}} = \mathbb{E}_{x_0, \epsilon \in \mathcal{N}(0,1), t} [\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2] .$$

The generation pipeline starts from noise $\epsilon_i \in \mathcal{N}(0, 1)$, diffused t times: $x_{\text{new}} = \epsilon_\theta(\dots(\epsilon_\theta(\epsilon_i, t), t-1), \dots, 2), 1)$.

Hyperparameters

The only hyperparameter tuned was the learning rate (visually), the other ones were chosen to be able to run this model on the machine (the model was very big, so I couldn't play very much with the hyperparameters).